

Numerical Method - Homework 2

B13902022 Yu-Chi,Lai

1

1.1 (a)

In section 3.4, I found: "Trivially, you can also apply a positive operator to a numeric expression: `int x = +42`; Numeric values are assumed to be positive unless explicitly made negative, so this operator has no effect on program operation"

Hence, the expression 0.0 conveys +0.0 by default. The code and its output is my experiment.

```
1  #include <stdio.h>
2  #include <stdint.h>
3  #include <string.h>
4
5  void print_bits(double d, const char* label) {
6      uint64_t bits;
7      memcpy(&bits, &d, sizeof(bits));
8      printf("%s: %f | Raw Hex: 0x%016lx\n", label, d, bits);
9  }
10
11 int main() {
12     double pos_zero = 0.0, neg_zero = -0.0;
13
14     print_bits(pos_zero, "Literal 0.0  ");
15     print_bits(neg_zero, "Literal -0.0 ");
16
17     if (pos_zero == neg_zero) {
18         printf("\nResult: The == operator says they are equal.\n");
19     }
20
21     printf("1 / 0.0 = %f\n", 1.0 / pos_zero);
22     printf("1 / -0.0 = %f\n", 1.0 / neg_zero);
23
24     return 0;
25 }
```

```
Literal 0.0  : 0.000000 | Raw Hex: 0x0000000000000000
Literal -0.0 : -0.000000 | Raw Hex: 0x8000000000000000

Result: The == operator says they are equal.
1 / 0.0 = inf
1 / -0.0 = -inf
```

Figure 1: The output

1.2 (b)

In section 7.12.3.6, there's an `signbit()` macro in C99 standard (And you should also `#include <math.h>`). The `signbit` macro returns a nonzero value if and only if the sign of its argument value is negative.

1.3 (c)

We can declare/specify them using unary operators, positive/negative number division by infinity, zero times positive/negative number or the `copysign()` method in `<math.h>`. The program below is my experiment:

```

1  #include <stdio.h>
2  #include <math.h>
3
4  int main() {
5      double values[] = {-0.0, 0.0, -1.0 / INFINITY, 1.0 / INFINITY, -1.0 * 0.0, 1.0 * 0.0,
6          ↪ copysign(0.0, 1.0), copysign(0.0, -1.0)};
7
8      const char *names[] = {"Literal -0.0", "Literal 0.0", "-1.0 / INFINITY", "1.0 /
9          ↪ INFINITY", "-1.0 * 0.0", "1.0 * 0.0", "copysign(0, +)", "copysign(0, -)"};
10
11     int n = sizeof(values) / sizeof(values[0]);
12
13     printf("Method          | Value | signbit()\n");
14     printf("-----|-----|-----\n");
15
16     for (int i = 0; i < n; i++) {
17         printf("%-18s | %4.1f | %d\n", names[i], values[i], signbit(values[i]));
18     }
19
20     printf("\n--- Validation through Division ---\n");
21     printf("1.0 / +0.0 = %f\n", 1.0 / values[1]);
22     printf("1.0 / -0.0 = %f\n", 1.0 / values[0]);
23
24     return 0;
25 }
```

Method	Value	signbit()
Literal -0.0	-0.0	1
Literal 0.0	0.0	0
-1.0 / INFINITY	-0.0	1
1.0 / INFINITY	0.0	0
-1.0 * 0.0	-0.0	1
1.0 * 0.0	0.0	0
copysign(0, +)	0.0	0
copysign(0, -)	-0.0	1

--- Validation through Division ---

1.0 / +0.0 = inf

1.0 / -0.0 = -inf

Figure 2: The output of the program above.

1.4 (d)

We should adopt the first implementation $((x > y)?x : y)$ because if we set $b = -0.0$, the returned value of the first implementation is $+0.0=0.0$ (Hence, $c = -\infty < 0$) while the second implementation gives -0.0 (Hence, $c = +\infty > 0$). We can use the program below to prove our claim.

```

1  #include <stdio.h>
2  #include <float.h>
3
4  double max1(double x, double y) {
5      return (x > y) ? x : y;
6  }
7
8  double max2(double x, double y) {
9      return (x < y) ? y : x;
10 }
11
12 int main() {
13     double a = -5, b = -0.0;
14     double c1 = a / max1(b, 0.0);
15     double c2 = a / max2(b, 0.0);
16     printf("%f %f\n", c1, c2);
17 }
```



Figure 3: The output.

2

(i)

Because I don't have matlab locally, I use GNU octave to implement everything.

No loop (vectorized) version:

```

1  function L = chol_vectorized(A)
2      n = size(A, 1);
3      L = zeros(n);
4      for k = 1:n
5          alpha = A(k, k);
6          L(k, k) = sqrt(alpha);
7          if (k < n)
8              v = A(k+1:n, k);
9              L(k+1:n, k) = v / L(k, k);
10             A(k+1:n, k+1:n) = A(k+1:n, k+1:n) - (v * v') / alpha;
11         end
12     end
13 end
```

One-level loop version:

```

1 function L = chol_onelevel(A)
2     n = size(A, 1);
3     L = zeros(n);
4     for k = 1:n
5         alpha = A(k, k);
6         L(k, k) = sqrt(alpha);
7         if k < n
8             v = A(k+1:n, k);
9             L(k+1:n, k) = v / L(k, k);
10            for i = 1:n-k
11                A(k+1:n, k+i) = A(k+1:n, k+i) - v * (v(i) / alpha);
12            end
13        end
14    end
15 end

```

Two-level loop version:

```

1 function L = chol_twolevel(A)
2     n = size(A, 1);
3     L = zeros(n);
4     for k = 1:n
5         alpha = A(k, k);
6         L(k, k) = sqrt(alpha);
7         if k < n
8             v = A(k+1:n, k);
9             L(k+1:n, k) = v / L(k, k);
10            for i = 1:n-k
11                for j = 1:n-k
12                    A(k+i, k+j) = A(k+i, k+j) - (v(i) * v(j)) / alpha;
13                end
14            end
15        end
16    end
17 end

```

(ii)

In my code, I generate a random $n \times n$ matrix X , and then we get a symmetry matrix XX^T which is also positive semi-definite. The reason why it is can be shown below:

$$\begin{aligned}
 \text{Let } A &= XX^T \\
 A^T &= (XX^T)^T = (X^T)^T X^T = XX^T = A \\
 \forall v \in \mathbb{R}^n, v \neq \vec{0}, v^T A v &= v^T A A^T v = (A^T v)^T (A^T v) \geq 0
 \end{aligned}$$

And then by adding a $n \times n$ identity matrix times a positive constant, we can ensure that $v^T (A + nI)v = v^T A v + n v^T I v > 0$, i.e., positive definite, without breaking the symmetry. The below is my code:

```

1 function A = generate_spd(n)
2     X = randn(n);
3     A = X * X' + n * eye(n);

```

```
4 endfunction
```

```
5
```

```
octave:1> A=generate_spd(4)
A =
    14.3011    0.1379   -1.6811    1.7991
     0.1379    8.4403    0.4961    0.1849
    -1.6811    0.4961    4.3781   -0.5137
     1.7991    0.1849   -0.5137    6.5175

octave:2> B=chol_vectorized(A), C=chol_onelevel(A), D=chol_twolevel(A), ans=chol(A, "lower")
B =
     3.7817         0         0         0
     0.0365     2.9050         0         0
    -0.4445     0.1764     2.0370         0
     0.4757     0.0577    -0.1534     2.5029

C =
     3.7817         0         0         0
     0.0365     2.9050         0         0
    -0.4445     0.1764     2.0370         0
     0.4757     0.0577    -0.1534     2.5029

D =
     3.7817         0         0         0
     0.0365     2.9050         0         0
    -0.4445     0.1764     2.0370         0
     0.4757     0.0577    -0.1534     2.5029

ans =
     3.7817         0         0         0
     0.0365     2.9050         0         0
    -0.4445     0.1764     2.0370         0
     0.4757     0.0577    -0.1534     2.5029
```

Figure 4: The correctness of my three implementations of cholesky factorization (proved by built-in `chol()`)

(iii)

I used meow1 (with GPU) in CSIE workstation, and the code for testing and its result are shown as below. For small to medium matrix sizes (such as $n = 10, 100, 1000$), the vectorized version is faster than the one-loop version, while the two-loop version is much slower. However, for $n = 4000$, the performance order changes to one-loop > vectorized > two-loop. Therefore, the fastest method depends on matrix size, and for very large matrices the one-loop version can outperform the vectorized implementation.

```
1 n_values = [10, 100, 1000, 4000];
2
3 for n = n_values
4     A = generate_spd(n);
5
6     % Measure Vectorized Version
7     tic;
8     chol_vectorized(A);
9     time_vec = toc;
10
11    % Measure One-level Loop Version
12    tic;
13    chol_onelevel(A);
14    time_one = toc;
15
16    % Measure Two-level Loop Version
17    tic;
18    chol_twolevel(A);
19    time_two = toc;
20
21    fprintf('For n=%d: Vectorized = %f s, One-loop = %f s, Two-loop = %f s\n', n, time_vec,
    ↪    time_one, time_two);
```

22 `end`

```

octave:1> test_time
For n=10: Vectorized = 0.002615 s, One-loop = 0.002170 s, Two-loop = 0.004068 s
For n=100: Vectorized = 0.006838 s, One-loop = 0.107226 s, Two-loop = 2.973830 s
For n=1000: Vectorized = 4.751259 s, One-loop = 7.543544 s, Two-loop = 2306.110248 s
For n=4000: Vectorized = 1137.850303 s, One-loop = 589.828728 s, Two-loop = 149219.219810 s
octave:2>

```

[0] 0:octave-cli-11.1.0* "meow1" 08:48 15-Mar-26

Figure 5: The results

(iv)

I found that the vectorized implementation significantly outperformed the looped versions because it avoids the interpreter overhead associated with element-by-element processing.

According to the Octave manual, vectorization allows the program to bypass high-level interpretation and utilize optimized internal C++, C, or Fortran libraries that leverage hardware vector instructions. The goal of vectorization is to write code that is making loops less (like for and while), instead, use whole array operations which is more efficient.

For small to medium matrices, this approach can achieve the typical speedup of Octave's matrix-oriented design, because vectorized operations call optimized low-level BLAS/LAPACK routines and reduce interpreter overhead.

However, for very large matrices (for example, $n = 4000$ in my test), the vectorized version became slower than the one-loop version. This is mainly due to memory pressure: the vectorized rank-1 updates create large temporary arrays repeatedly, which increases memory traffic and cache misses. The one-loop version updates matrix columns directly, which is more cache-friendly and reduces temporary allocation cost.

The two-loop version is still the slowest because it performs element-by-element updates, so loop control and interpreter overhead accumulate heavily. In summary, the Octave results are consistent with the MATLAB analysis: vectorization is best for smaller sizes, while one-loop can be better for large-scale problems where memory bandwidth becomes the bottleneck.

3 Matlab implementation (2(v))

The behind-scene theory and technique I used in this part is almost the same as that in octave, so I don't further explain how these codes work here.

3.1 Symmetric positive definite matrix

```

1 function A = generate_spd(n)
2     % Generate an n x n symmetric positive definite matrix
3     X = randn(n);
4     A = X * X' + n * eye(n);
5 end

```

Figure 6: The function for generating Symmetric positive definite matrix (using MATLAB)

```

>> A=generate_spd(10)

A =

    18.3593    2.3619   -2.8169    1.0698   -2.3020   -1.9120   -1.3181   -0.1777    1.3942   -1.9039
    2.3619    18.5100   -2.9416    3.2594   -8.0547    2.5948   -1.5456    2.3472   -4.5755   -2.1081
   -2.8169   -2.9416    20.1710    2.6296    6.7800   -8.6171   -0.3699   -2.3061    1.5395    3.4958
    1.0698    3.2594    2.6296   17.5775   -3.6336   -5.0278   -2.6246    0.9986   -3.3293    0.0090
   -2.3020   -8.0547    6.7800   -3.6336   27.6678   -7.6917    3.7774   -3.8755    1.3230    4.4883
   -1.9120    2.5948   -8.6171   -5.0278   -7.6917   30.1782    2.1851    0.2323   -3.4919   -7.2710
   -1.3181   -1.5456   -0.3699   -2.6246    3.7774    2.1851   14.2112    0.9743    2.7121    1.2961
   -0.1777    2.3472   -2.3061    0.9986   -3.8755    0.2323    0.9743   17.3681    1.9696    3.9248
    1.3942   -4.5755    1.5395   -3.3293    1.3230   -3.4919    2.7121    1.9696   25.7654   -0.5608
   -1.9039   -2.1081    3.4958    0.0090    4.4883   -7.2710    1.2961    3.9248   -0.5608   24.7135

>> eig(A)

ans =

    10.0985
    11.4028
    12.4328
    13.2348
    14.5490
    18.6221
    24.9283
    27.3137
    32.4736
    49.4664

```

Figure 7: By using `eig()` function, we can found all eigenvalues are positive for real.

3.2 Cholesky decomposition (3 methods)

```

1 function L = chol_vectorized(A)
2     n = size(A, 1);
3     L = zeros(n);
4     for k = 1:n
5         alpha = A(k,k);
6         L(k,k) = sqrt(alpha);
7         if k < n
8             v = A(k+1:n, k);
9             L(k+1:n, k) = v / L(k,k);
10            % Vectorized update: No inner loops
11            A(k+1:n, k+1:n) = A(k+1:n, k+1:n) - (v * v') / alpha;
12        end
13    end
14 end

```

Figure 8: Vectorized Cholesky decomposition implementation (using MATLAB)

```

1 function L = chol_oneloop(A)
2     n = size(A, 1);
3     L = zeros(n);
4     for k = 1:n
5         alpha = A(k,k);
6         L(k,k) = sqrt(alpha);
7         if k < n
8             v = A(k+1:n, k);
9             L(k+1:n, k) = v / L(k,k);
10            % One-level loop: Update each column j
11            for j = k+1:n
12                A(k+1:n, j) = A(k+1:n, j) - v * (v(j-k) / alpha);
13            end
14        end
15    end
16 end

```

Figure 9: One-loop Cholesky decomposition implementation (using MATLAB)

```

1 function L = chol_twoloop(A)
2     n = size(A, 1);
3     L = zeros(n);
4     for k = 1:n
5         alpha = A(k,k);
6         L(k,k) = sqrt(alpha);
7         if k < n
8             v = A(k+1:n, k);
9             L(k+1:n, k) = v / L(k,k);
10            % Two-level loop: Update each element B(i,j)
11            for j = k+1:n
12                for i = k+1:n
13                    A(i, j) = A(i, j) - v(i-k) * (v(j-k) / alpha);
14                end
15            end
16        end
17    end
18 end

```

Figure 10: Two-loop Cholesky decomposition implementation (using MATLAB)


```

>> A=generate_spd(3)

A =

    4.7066   -1.1580    0.0079
   -1.1580   13.9756    1.0684
    0.0079    1.0684    4.9104

>> chol(A, "lower")

ans =

    2.1695         0         0
   -0.5338    3.7001         0
    0.0037    0.2893    2.1970

>> chol_vectorized(A)

ans =

    2.1695         0         0
   -0.5338    3.7001         0
    0.0037    0.2893    2.1970

>> chol_one_loop(A)

ans =

    2.1695         0         0
   -0.5338    3.7001         0
    0.0037    0.2893    2.1970

>> chol_two_loop(A)

ans =

    2.1695         0         0
   -0.5338    3.7001         0
    0.0037    0.2893    2.1970

```

Figure 11: The correctness of my three codes by `chol()` function

3.3 Efficiency analysis

I tested the three Cholesky factorization implementations in MATLAB (using the MATLAB Online free plan) for $n \in \{10, 100, 1000, 2000, 4000\}$. The timing results revealed a significant shift in performance efficiency as the matrix size n increased.

For small to medium matrices ($n \leq 1000$), the **Vectorized** version was the fastest. Because the matrices are relatively small, the memory required to perform the operations easily fits into the CPU's high-speed L1/L2 cache. Consequently, the vectorized code fully benefits from MATLAB's underlying highly-optimized numerical libraries (such as BLAS and LAPACK) and bypasses the interpreter overhead associated with loop management.

However, for massive matrices ($n = 4000$), the performance inverted entirely: the **One-loop** version was the fastest (≈ 37.1 s), the **Two-loop** version was second (≈ 65.8 s), and the **Vectorized** version was the slowest by a wide margin (≈ 113.7 s). This occurs due to the interplay between memory bandwidth and compiler optimization:

- **Why One-loop outperforms Vectorized (The Memory Bottleneck):** The vectorized update formula computes the outer product vv^T , generating a massive temporary matrix (up to 4000×4000) during

every single iteration. Continuously allocating, writing, and clearing this large block of memory forces the CPU to constantly fetch data from the slower main RAM, causing severe cache thrashing. The One-loop version avoids this by updating the matrix column-by-column. A single column easily fits into the CPU cache, eliminating the memory bandwidth bottleneck.

- **Why One-loop outperforms Two-loop (JIT and Interpreter Overhead):** While both looped versions avoid creating massive temporary matrices, the Two-loop updates the matrix element-by-element. Even though modern MATLAB features a Just-In-Time (JIT) compiler to accelerate loops, managing the loop conditions and array bounds for millions of individual $A(i, j)$ elements still introduces noticeable computational overhead. The One-loop version hits the optimal "sweet spot": it processes cache-friendly chunks of data (columns) while still utilizing low-level, optimized vector operations for each column update.

```

1  % Define the test sizes
2  n_values = [10, 100, 1000, 2000, 4000];
3
4  for n = n_values
5      fprintf('--- Testing for n = %d ---\n', n);
6      A = generate_spd(n);
7
8      % Validation
9      L_builtin = chol(A, 'lower');
10     L_vec = chol_vectorized(A);
11     error_norm = norm(L_builtin - L_vec, 'fro');
12     fprintf('Validation Error: %e\n', error_norm);
13
14
15     tic;
16     chol_vectorized(A);
17     t_vec = toc;
18     fprintf('Vectorized time: %f seconds\n', t_vec);
19
20     tic;
21     chol_oneloop(A);
22     t_one = toc;
23     fprintf('One-loop time:    %f seconds\n', t_one);
24
25     tic;
26     chol_twoloop(A);
27     t_two = toc;
28     fprintf('Two-loop time:    %f seconds\n', t_two);
29
30     fprintf('\n');
31 end

```

Figure 12: Performance testing script for Matlab implementations (using MATLAB)

```
>> test_time_matlab
--- Testing for n = 10 ---
Validation Error: 2.674007e-16
Vectorized time: 0.008201 seconds
One-loop time: 0.002512 seconds
Two-loop time: 0.001902 seconds

--- Testing for n = 100 ---
Validation Error: 2.501257e-14
Vectorized time: 0.001380 seconds
One-loop time: 0.004725 seconds
Two-loop time: 0.003524 seconds

--- Testing for n = 1000 ---
Validation Error: 7.496909e-13
Vectorized time: 0.820257 seconds
One-loop time: 0.762828 seconds
Two-loop time: 0.988979 seconds

--- Testing for n = 2000 ---
Validation Error: 2.063336e-12
Vectorized time: 5.416832 seconds
One-loop time: 4.211101 seconds
Two-loop time: 7.920561 seconds

--- Testing for n = 4000 ---
Validation Error: 5.912315e-12
Vectorized time: 113.700316 seconds
One-loop time: 37.168590 seconds
Two-loop time: 65.825860 seconds
```

Figure 13: The execution time tests for Vectorized, One-loop, Two-loop version of Cholesky decomposition